

AEB - Academic Evolutionary Biosphere

学术演化生物圈

完整技术手册与用户指南

Complete Technical Manual & User Guide

Version 2.0.0

基于提供的ZIP压缩包的详尽源代码审计

Based on exhaustive source code audit of the provided ZIP archive

Item	Detail
Entry Point	app.py (Streamlit multi-page application)
Total Pages	13 functional pages + 1 main overview
Core Engine	utils/data_engine.py (TF-IDF, NetworkX, Louvain)
AI Backends	GGUF (llama-cpp) / Ollama / OpenAI-compatible
3D Rendering	Three.js r128 via streamlit.components.v1
Language	Bilingual Chinese/English (i18n module)
Python	>= 3.9 (developed on 3.11)

Table of Contents / 目录

- 0. Executive Summary / 执行摘要
- 1. Project Structure & Module Map / 项目结构与模块映射
- 2. Formula Registry / 公式注册表
- 3. System Mechanism Diagram / 系统机制图
- 4. Methodology / 方法论
- 5. Computation Workflow / 计算 workflow
- 6. FAQ & Troubleshooting / 常见问题与故障排除
- 7. Quick Start Guide / 快速入门指南
- 8. Supplementary Notes / 补充说明
- Appendix A: Dependency List / 依赖列表
- Appendix B: Configuration Reference / 配置参考
- Appendix C: i18n Key Reference / 国际化键名参考

0. Executive Summary / 执行摘要

0.1 What Is This System / 本系统简介

AEB (Academic Evolutionary Biosphere) is a Streamlit-based multi-page web application that treats academic keyword ecosystems as biological ecosystems. It ingests academic papers in multiple formats, extracts keywords via TF-IDF, builds co-occurrence networks, computes ecological metrics (Shannon diversity, PageRank, betweenness centrality, Louvain community detection), and presents the results through 13 interactive visualization pages including 3D renderings (Three.js), Plotly charts, and optional AI-powered natural language analysis.

0.2 Problem It Solves / 解决的问题

Input: A collection of academic papers (txt, pdf, md, docx, json, csv, xlsx, html, xml, yaml). Processing: Keyword extraction (TF-IDF), co-occurrence matrix construction, network graph analysis (centrality, PageRank, Louvain clustering), ecological role classification (top_predator / dominant / symbiotic), Shannon diversity and monopoly ratio computation, evolution tree construction, niche level assignment, carbon sink calculation, extinction detection. Output: 13 interactive visualization pages, AI analysis reports, and multi-format exports (JSON, CSV, TXT, ZIP).

0.3 Core Capabilities / 核心能力概览

- Multi-format file parsing (15+ formats) with encoding auto-detection and fallback chain
- TF-IDF keyword extraction via sklearn TfidfVectorizer
- Co-occurrence matrix construction and keyword network graph building (NetworkX)
- Betweenness centrality and PageRank computation
- Louvain community detection (python-louvain, with greedy-modularity fallback)
- Ecological role classification: top_predator (top 1%), dominant (top 10%), symbiotic (rest)
- Shannon diversity index and monopoly ratio (top-3 concentration) computation
- Keyword extinction detection based on temporal decay analysis
- BFS-based evolution tree construction from co-occurrence relationships
- Niche level assignment (5 levels via quantile binning)
- Carbon sink = total co-occurrence weight per keyword
- 3D visualizations: Evolution Tree, Force-directed Graph, Fossil Layers, Migration Paths, Niche Hierarchy (all Three.js r128)
- 2D Plotly visualizations: Fitness curves, extinction donut/bar, sedimentation rate, treemap, scatter clusters
- Three AI backends: llama-cpp-python (local GGUF), Ollama, OpenAI-compatible API
- Bilingual Chinese/English i18n with runtime toggle
- Per-page export: JSON, CSV, TXT, ZIP with audit log
- Species collection system: custom keyword bookmarks with Jaccard overlap comparison

0.4 Runtime Requirements / 运行要求

Python >= 3.9 (developed on 3.11). Required: streamlit >= 1.30, pandas >= 2.0, numpy >= 1.24, scikit-learn >= 1.3, networkx >= 3.1, python-louvain >= 0.16, scipy >= 1.11, PyYAML >= 6.0, plotly >= 5.18,

PyMuPDF >= 1.23, pdfplumber >= 0.10, python-docx >= 1.0, beautifulsoup4 >= 4.12, lxml >= 4.9, openpyxl >= 3.1, chardet >= 5.2. Optional AI: ollama >= 0.3, openai >= 1.10, llama-cpp-python >= 0.2.50.

0.5 Confirmed Limitations / 已知限制

`first_year` defaults to 2000 for all extracted keywords (line in `extract_keywords_from_texts`); actual per-keyword year data is not computed from `papers_df` year field -- the TODO comment says 'override with actual data' but this is never done.

Extinction detection (`detect_extinction`) is defined but never called in `compute_all()`; `year_counts` remains empty. The extinction UI page (04) uses a simplistic age-based heuristic instead.

No random seed control; co-occurrence and network layout are deterministic, but AI outputs and migration path positions use `random.uniform` without seeding.

No persistent storage; all state lives in `st.session_state` and is lost on page refresh.

No unit tests exist in the codebase.

No authentication, rate limiting, or data privacy controls.

Force simulation in `Three.js` runs for fixed 250 frames regardless of graph convergence quality.

`n_gpu_layers=-1` defaults may cause VRAM OOM on smaller GPUs when loading large GGUF models.

1. Project Structure & Module Map / 项目结构与模块映射

1.1 File Tree / 文件树

```
aeb-project-v3/
|-- app.py                # Main entry: Streamlit homepage + sidebar
|-- config.yaml           # Global YAML configuration
|-- requirements.txt       # pip dependencies
|-- run.bat               # Windows launch script (port 7861)
|-- install.bat           # Windows dependency installer
|-- README.md             # Project documentation
|-- pages/
| |-- 01_evolution_tree.py # 3D evolution tree (Three.js)
| |-- 02_neuron_graph.py   # Force-directed keyword network (Three.js)
| |-- 03_fossil_layers.py  # Geological fossil layer visualization
| |-- 04_extinction_sim.py # Extinction event simulation (Plotly)
| |-- 05_fitness_curve.py  # Keyword fitness over time (Plotly)
| |-- 06_migration_path.py # Cross-domain migration animation (Three.js)
| |-- 07_origin_trace.py   # Reverse ancestry trace (Plotly)
| |-- 08_carbon_sink.py    # Knowledge contribution treemap (Plotly)
| |-- 09_niche_cluster.py  # Louvain community scatter (Plotly)
| |-- 10_sedimentation.py  # Publication rate monitor (Plotly)
| |-- 11_fossil_report.py  # Deep-dive single keyword report
| |-- 12_niche_hierarchy.py # 5-level niche hierarchy (Three.js)
| |-- 13_collection.py     # Custom keyword bookmarks + comparison
| |-- __init__.py
|-- utils/
| |-- aeb_model.py         # AI backend manager (GGUF/Ollama/OpenAI)
| |-- data_engine.py       # Core computation engine
| |-- chart_theme.py       # Plotly theme + color palettes
| |-- threejs_components.py # Three.js HTML component renderers
| |-- file_parser.py       # Multi-format file parser
| |-- export_utils.py      # JSON/CSV/TXT/ZIP export helpers
| |-- i18n.py              # Internationalization (zh/en)
| |-- __init__.py
```

1.2 Entry Point & Launch / 入口与启动

Primary entry: app.py. Streamlit's multi-page convention auto-discovers pages/ directory. Launch command:

```
streamlit run app.py --server.port 7861 --server.fileWatcherType poll
```

Alternative: double-click run.bat (Windows). Install dependencies first via: pip install -r requirements.txt (or double-click install.bat on Windows).

1.3 Module Responsibility Table / 模块职责表

Module	Responsibility	Key Functions/Classes	Called By
app.py	Main page: sidebar (upload, settings, model panel), overview metrics, top predator/dominant/cluster tabs, AI analysis	load_config(), main flow	Streamlit entry

Module	Responsibility	Key Functions/Classes	Called By
data_engine.py	All keyword extraction, network analysis, ecological classification, diversity metrics, evolution tree, singleton engine	AEBDDataEngine, extract_keywords_from_texts, build_cooccurrence_matrix, build_network, compute centrality, compute_pagerank, classify_ecology, compute_shannon_index, compute_monopoly_ratio, detect_extinction, compute_niche_levels, compute_carbon_sink, build_evolution_tree, detect_louvain_communities, get_engine	app.py, all 13 pages
aeb_model.py	AI model lifecycle: detect, load, generate, health check, stream; sidebar model config UI	AEBModelManager, LlamaCppBackend, OllamaBackend, OpenAIBackend, render_model_panel, ai_analyze, render_ai_panel, get_model_manager	app.py, all 13 pages

Module	Responsibility	Key Functions/Classes	Called By
file_parser.py	Parse uploaded files (15+ formats) with encoding detection and fallback chain	parse_file, parse_multiple_files, _parse_pdf, _parse_docx, _parse_excel, etc.	app.py (sidebar upload)
chart_theme.py	Plotly dark theme, color palettes (PALETTE_NEON, ROLE_COLORS, etc.)	apply_theme, AEB_LAYOUT	All Plotly pages
threejs_component_s.py	Generate self-contained HTML+JS for Three.js 3D scenes; 5 renderers	render_force_graph, render_evolution_tree, render_fossil_layers, render_migration_paths, render_niche_hierarchy, _wrap_html	Pages 01,02,03, 06,12
export_utils.py	Download buttons for JSON, CSV, TXT, ZIP (with audit log)	export_json, export_csv, export_txt, export_zip, render_export_panel	app.py, all 13 pages
i18n.py	Bilingual key-value lookup, language init/toggle	t(), init_lang(), toggle_lang(), TEXTS dict	All files

1.4 Data Flow Summary / 数据流概要

1) User uploads files via sidebar file_uploader (15+ format types). 2) On 'Start Loading', parse_multiple_files() invokes file_parser.py which returns {text, success, format, error, filename} per file. 3) engine.ingest() stores raw texts and extracts years via regex from the first 2000 chars. 4) engine.compute_all() runs the 8-step pipeline: TF-IDF extraction -> co-occurrence matrix -> network graph -> centrality + PageRank -> ecology classification + niche levels + carbon sink -> Shannon index + monopoly ratio -> Louvain community detection -> evolution tree BFS. 5) All computed state is stored in AEBDataEngine fields (DataFrames, dicts, nx.Graph). 6) Each page reads from get_engine() singleton and renders its specific visualization. 7) AI analysis is triggered per-page via button; prompt is constructed from engine data and sent to the active backend via ai_analyze(). 8) Exports are generated on-demand as download buttons (JSON/CSV/TXT/ZIP) via render_export_panel().

2. Formula Registry / 公式注册表

2.1 Discovery Rules / 发现规则说明

All formulas below were identified from explicit code expressions, sklearn/networkx/scipy library calls, and hand-written calculations in `utils/data_engine.py`, `pages/*.py`, and `utils/threejs_components.py`. Library-implemented formulas include their standard mathematical form annotated with the implementing function.

F-001: TF-IDF (Term Frequency - Inverse Document Frequency)

$$\text{tfidf}(t,d,D) = \text{tf}(t,d) * \text{idf}(t,D)$$

where $\text{tf}(t,d) = \text{count}(t \text{ in } d)$, $\text{idf}(t,D) = \log((1+|D|) / (1+\text{df}(t))) + 1$

Then L2-normalized per document row.

Final metric: $\text{tfidf_mean}(t) = \text{mean over all documents } d \text{ in } D \text{ of } \text{tfidf}(t,d)$

Location: `data_engine.py :: extract_keywords_from_texts()`. Implemented by:

`sklearn.feature_extraction.text.TfidfVectorizer` (default `sublinear_tf=False`, `smooth_idf=True`, `norm='l2'`).

Parameters: `max_features=top_n*3`, `min_df=min(min_df, len(texts))`, `max_df=0.95`, `stop_words='english'`, `token_pattern=r'(?u)\b[a-zA-Z\u4e00-\u9fff]{2,}\b'`. Variable definitions: t =term/keyword, d =document text, D =corpus of all texts, $|D|$ =number of documents, $\text{df}(t)$ =number of documents containing t . Purpose: Primary keyword importance scoring; foundation for all downstream ecological metrics. Numerical stability: sklearn internally handles $\log(0)$ via `smooth_idf=True` (+1 in denominator). Complexity: $O(|D| * V)$ where V = vocabulary size. Called once during `compute_all()`.

F-002: Co-occurrence Count

$$\text{cooccur}(a, b) = |\{d \text{ in } D : a \text{ in } d \text{ AND } b \text{ in } d\}|$$

For each document d , extract unique keywords present, then increment $\text{cooccur}(a,b)$ for all pairs. Location: `data_engine.py :: build_cooccurrence_matrix()`. Hand-written nested loop over keyword pairs in each document. Variable definitions: a,b = keywords; d = document text; D = corpus. Purpose: Edge weights in the keyword network graph; basis for carbon sink calculation. Numerical stability: Integer counts, no floating point issues. Complexity: $O(|D| * K^2)$ where K = number of keywords per document. Called once.

F-003: Betweenness Centrality

$$C_B(v) = \text{SUM over all } s \neq v \neq t \text{ of } (\text{sigma_st}(v) / \text{sigma_st}(s,t))$$

where sigma_st = number of shortest paths from s to t , $\text{sigma_st}(v)$ = number of those paths passing through v . Location: `data_engine.py :: compute_centrality()`. Implemented by:

`networkx.betweenness_centrality(G, weight='weight')`. Uses Brandes' algorithm. Variable definitions: v = node (keyword), s,t = source/target nodes in graph G . Purpose: Measures how often a keyword bridges different parts of the network; contributes to 'score'. Numerical stability: networkx handles disconnected components (returns 0 for isolated nodes). Complexity: $O(V * E)$ where V =nodes, E =edges. Called once. For 200 keywords this is fast.

F-004: PageRank

$$\text{PR}(v) = (1-d)/N + d * \text{SUM over } u \text{ in neighbors}(v) \text{ of } (\text{PR}(u) * w(u,v) / \text{SUM}_j w(u,j))$$

where d = damping factor (default 0.85), N = number of nodes. Location: `data_engine.py ::`

`compute_pagerank()`. Implemented by: `networkx.pagerank(G, weight='weight')`. Variable definitions: v = keyword node, d = damping (0.85 default), $w(u,v)$ =

co-occurrence edge weight. Purpose: Alternative importance ranking; stored per keyword but not directly used in ecological classification (score = tfidf_mean + centrality, not PageRank). Displayed in UI tables. Fallback: On exception, returns uniform $1/N$ for all nodes. Complexity: $O(E * \text{iterations})$, typically converges in 20-100 iterations. Called once.

F-005: Composite Ecological Score

$\text{score}(k) = \text{tfidf_mean}(k) + \text{centrality}(k)$

where centrality = betweenness centrality from F-003. Location: `data_engine.py :: classify_ecology()` and `compute_all()`. Hand-written addition. Variable definitions: k = keyword. Purpose: Combined metric for ranking keywords; determines ecological role assignment. Note: `tfidf_mean` and `centrality` operate on different scales (`tfidf` typically 0-0.1, `centrality` 0-1); no normalization is applied. This means centrality tends to dominate for well-connected keywords. Risk: Scale imbalance may bias classification. No eps or clipping applied.

F-006: Ecological Role Classification

$\text{role}(k) = \text{top_predator}$ if $\text{rank}(k) < \max(1, N * \text{top_pct})$
 $= \text{dominant}$ if $\text{rank}(k) < \max(2, N * \text{dom_pct})$
 $= \text{symbiotic}$ otherwise

where rank is by score descending, `top_pct=0.01`, `dom_pct=0.10` (from config). Location: `data_engine.py :: classify_ecology()`. Hand-written index-based classification. Variable definitions: N = total keywords, `top_pct` = `top_predator_percentile` (config: 0.01), `dom_pct` = `dominant_percentile` (config: 0.10). Purpose: Assigns ecological metaphor roles for visualization coloring and filtering. Note: Uses rank index, not percentile of score values. `max(1,...)` ensures at least 1 `top_predator`.

F-007: Shannon Diversity Index

$H = - \sum_i (p_i * \log_2(p_i))$

where $p_i = \text{tfidf_mean}(\text{keyword}_i) / \sum_j \text{tfidf_mean}(\text{keyword}_j)$. Only terms with $p_i > 0$ are included. Location: `data_engine.py :: compute_shannon_index()`. Hand-written using numpy. `np.sum(proportions * np.log2(proportions))`. Variable definitions: p_i = proportion of keyword i 's TF-IDF in total; H = diversity in bits. Purpose: Measures keyword ecosystem diversity; low H triggers 'ecological collapse warning' if $H < \text{shannon_warning_threshold}$ (config: 1.5). Numerical stability: Filters $p_i > 0$ before log; returns 0.0 if total=0. No eps needed. Complexity: $O(K)$ where K = number of keywords.

F-008: Monopoly Ratio

$\text{monopoly} = \text{SUM}(\text{tfidf_mean of top 3 keywords}) / \text{SUM}(\text{tfidf_mean of all keywords})$

Location: `data_engine.py :: compute_monopoly_ratio()`. Hand-written ratio. Variable definitions: `top_n = 3` (hardcoded). Purpose: Measures concentration; displayed in sidebar. High ratio = keyword monopoly. Numerical stability: Returns 0.0 if total=0.

F-009: Carbon Sink

$\text{carbon_sink}(k) = \text{SUM over all } j \text{ of } \text{cooccur}(k, j)$

= row sum of the co-occurrence matrix for keyword k . Location: `data_engine.py :: compute_carbon_sink()`. Hand-written row sum of integer co-occurrence matrix. Purpose: Metaphor for 'knowledge contribution' -- keywords that co-occur with many others have high carbon sink. Visualized in Page 08 (treemap, bar chart). Complexity: $O(K)$ per keyword.

F-010: Niche Level Assignment

$\text{niche_level}(k) = \text{quantile_bin}(\text{rank}(\text{score}(k)), n_levels=5) + 1$

where `quantile_bin` uses `pd.qcut` with `method='first'` to break ties. Location: `data_engine.py :: compute_niche_levels()`. Implemented by: `pandas.qcut(..., labels=False, duplicates='drop') + 1`. Purpose: Assigns keywords to 5 tiers (L1 Foundation to L5 Domain Application). Visualized in Page 12 (3D niche hierarchy).

F-011: Louvain Community Detection

$Q = (1/2m) * \sum_{ij} [A_{ij} - (k_i * k_j)/(2m)] * \delta(c_i, c_j)$

where A_{ij} = edge weight, k_i = weighted degree, m = total edge weight/2, c_i = community of node i , δ = Kronecker delta. Louvain greedily maximizes Q with resolution parameter γ (default 1.0). Location: `data_engine.py :: detect_louvain_communities()`. Implemented by: `community.best_partition(G, weight='weight', resolution=config.louvain_resolution)`. Fallback: `networkx.community.greedy_modularity_communities`. Second fallback: all nodes assigned community 0. Purpose: Groups keywords into thematic clusters. Visualized in Pages 09, 02. Complexity: $O(E * \log V)$ typical for Louvain.

F-012: Force-Directed Graph Simulation (Three.js)

Repulsion: $F_{\text{rep}}(i,j) = 60 / \text{dist}(i,j)^2$ (applied to x,y components)

Attraction (edges): $F_{\text{attr}} = (\text{dist} - 50) * 0.004$ (spring-like, rest length=50)

Runs for 250 fixed frames then stops.

Location: `threejs_components.py :: render_force_graph()` (JavaScript inline). Hand-written in the JS `simStep()` function. Variable definitions: dist = Euclidean distance between node positions, 60 = repulsion constant, 50 = rest length, 0.004 = spring constant. Purpose: Layout for the neuron graph (Page 02) 3D visualization. Note: No convergence check; always runs 250 frames regardless of layout quality.

F-013: Extinction Detection (defined but not called)

$\text{extinct}(k) = \text{True}$ if $\text{peak_count} > 0$ AND $\text{recent_count} == 0$

OR $\text{peak_count} > 0$ AND $(\text{peak_count} - \text{recent_count})/\text{peak_count} > \text{decay_threshold}$

where $\text{recent_count} = \sum(\text{count}(k, y))$ for y in $[\text{current_year} - \text{lookback}, \text{current_year}]$, $\text{peak_count} = \max(\text{count}(k, y))$ over all years. Location: `data_engine.py :: detect_extinction()`. Hand-written conditional. Parameters: $\text{decay_threshold}=0.80$, $\text{lookback}=5$ (from config). IMPORTANT: This function is DEFINED but NEVER CALLED in `compute_all()`. `year_counts` is always empty dict. Page 04 (`extinction_sim.py`) uses a different, simpler age-based heuristic: $\text{age} > 20 = \text{extinct}$, $10 < \text{age} \leq 20 = \text{fading}$, else alive.

2.3 Formula Dependency Chain / 公式依赖链

Preprocessing: F-001 (TF-IDF) -> F-002 (Co-occurrence). Core computation: F-002 feeds into F-003 (Centrality), F-004 (PageRank), F-009 (Carbon Sink), F-011 (Louvain). F-001 + F-003 -> F-005 (Score) -> F-006 (Role Classification) -> F-010 (Niche Level). F-001 -> F-007 (Shannon Index), F-008 (Monopoly Ratio). Display only: F-012 (Force layout) uses F-002 edge weights for spring simulation. Unused: F-013 (Extinction detection) is dead code.

3. System Mechanism Diagram / 系统机制图

3.1 Mermaid Flowchart (copy-paste to any Mermaid renderer)

```
graph TD
  A[User uploads files] --> B[file_parser]
  B --> C{Parse success?}
  C -- Yes --> D[AEBDataEngine.ingest]
  C -- No --> E[Fallback: raw text decode]
  E --> D
  D --> F[compute_all pipeline]
  F --> F1[Step 1: TF-IDF extraction]
  F1 --> F2[Step 2: Co-occurrence matrix]
  F2 --> F3[Step 3: NetworkX graph]
  F3 --> F4[Step 4: Centrality + PageRank]
  F4 --> F5[Step 5: Ecology classification]
  F5 --> F6[Step 6: Shannon + monopoly]
  F6 --> F7[Step 7: Louvain communities]
  F7 --> F8[Step 8: BFS evolution tree]
  F8 --> G[AEBDataEngine singleton]
  G --> P01..P13[Pages 01-13]
  G --> AI[AI Analysis]
  G --> EX[Export (JSON/CSV/TXT/ZIP)]
```

3.2 Diagram Explanation / 图表说明

Node A: User uploads academic papers through Streamlit file_uploader (sidebar). Node B: file_parser.py routes each file to the correct parser by extension. Nodes C/E: Success path or fallback to raw text extraction with error logging. Node D: AEBDataEngine.ingest() stores texts and extracts publication years. Nodes F1-F8: The 8-step compute pipeline runs sequentially with progress callback. Node G: The singleton engine holds all computed state in session_state. Nodes P01-P13: Each Streamlit page reads from the engine and renders its visualization. Node AI: Per-page AI analysis sends constructed prompts to the active backend. Node EX: Each page includes download buttons for JSON, CSV, TXT, ZIP with audit logs.

3.3 Page-to-Backend Module Mapping / 页面到后端模块映射

Page	Backend Modules & Data Used	Render
01 Evolution Tree	data_engine (evolution_tree), threejs_components (render_evolution_tree)	Three.js 3D
02 Neuron Graph	data_engine (keywords_df, cooccurrence, communities), threejs_components (render_force_graph)	Three.js 3D
03 Fossil Layers	data_engine (keywords_df by year range), threejs_components (render_fossil_layers)	Three.js 3D
04 Extinction Sim	data_engine (keywords_df, papers_df years)	Plotly 2D
05 Fitness Curve	data_engine (get_fitness_data per keyword)	Plotly 2D
06 Migration Path	data_engine (get_keyword_neighbors), threejs_components (render_migration_paths)	Three.js 3D
07 Origin Trace	data_engine (get_keyword_neighbors, co-occurrence ancestor chain)	Plotly 2D
08 Carbon Sink	data_engine (carbon_sink column)	Plotly 2D
09 Niche Cluster	data_engine (community column, centrality, tfidf)	Plotly 2D

Page	Backend Modules & Data Used	Render
10 Sedimentation	data_engine (papers_df year counts)	Plotly 2D
11 Fossil Report	data_engine (single keyword deep-dive: fitness, neighbors, replacements)	Plotly 2D
12 Niche Hierarchy	data_engine (niche_level column), threejs_components (render_niche_hierarchy)	Three.js 3D
13 Collection	data_engine (keywords_df, compute_shannon_index), session_state collections	Table/Text

4. Methodology / 方法论

4.1 Problem Definition / 问题定义

Input: A set of academic papers in heterogeneous formats. Output: An interactive ecological visualization mapping keywords to biological metaphors (species, predators, niches, fossils, extinction events), augmented with optional AI narrative analysis. Objective: Enable researchers to understand the structure, diversity, health, and evolutionary dynamics of a research field's keyword ecosystem. Constraints: Must run locally on commodity hardware; AI features are optional (graceful degradation).

4.2 Overall Approach / 总体方法

The system is organized in three layers: (1) Data Ingestion Layer -- robust multi-format parsing with cascading fallbacks; (2) Computation Layer -- a deterministic 8-step pipeline producing ecological metrics from NLP and graph analysis; (3) Presentation Layer -- 13 specialized visualization pages using Three.js for 3D and Plotly for 2D, with optional AI narration.

4.3 Key Design Choices / 关键设计选择

Ecological metaphor framework: Keywords are treated as 'species' in an ecosystem. TF-IDF serves as 'biomass', centrality as 'bridging importance', co-occurrence as 'symbiosis', PageRank as 'influence'. This metaphor makes abstract bibliometric data intuitive.

TF-IDF + Betweenness Centrality composite score: Combines content importance (TF-IDF) with structural importance (centrality) for ecological role assignment. Deliberate choice over PageRank alone.

Louvain with greedy-modularity fallback: Provides robust community detection even when python-louvain is unavailable.

Three.js inline HTML components: Avoids npm build step; self-contained HTML strings rendered via streamlit.components.v1. Trades bundle size optimization for zero-config deployment.

Three-tier AI backend with priority chain: GGUF (fastest, offline) > Ollama (local, flexible) > OpenAI (remote, requires key). Graceful degradation: all features work without AI; AI adds narrative analysis only.

Singleton engine in session_state: Ensures computed data persists across page navigations within a session. Trade-off: data is lost on browser refresh.

Bilingual i18n via dictionary lookup: Simple, no external i18n framework dependency. All UI strings support zh/en toggle.

4.4 Robustness Strategy / 健壮性策略

File parsing: Each parser has a dedicated try/except; on failure, falls to `_fallback_text()` which does raw UTF-8 decode with `errors='ignore'` and strips non-printable characters. Encoding detection uses chardet with a 5-encoding fallback chain (utf-8, gbk, gb2312, latin1, ascii). Network analysis: `compute_centrality` and `compute_pagerank` each have except blocks returning zeros/uniform on failure. Louvain has a two-level fallback chain. AI: `AEBModelManager.generate` wraps all calls in try/except, returning error messages as strings. The UI displays a 'degraded mode' banner when no AI backend is connected. Empty data: Every page checks `engine.is_computed` and `keywords_df.empty`, showing 'no_data' message and calling `st.stop()` early.

4.5 Reproducibility & Audit / 可复现性与审计

Confirmed present: Export ZIP includes `audit_log.txt` with timestamp and file list. `config.yaml` stores all tunable ecological parameters. Deterministic pipeline (sklearn TF-IDF, networkx centrality/PageRank are deterministic for same input). Confirmed missing: No random seed control for migration path

random.uniform() positions. No version pinning in requirements.txt (uses >= operators only). No run log, no checkpointing, no git hash recording. Minimum augmentation recommended: (1) Pin exact dependency versions. (2) Add random seed parameter to config.yaml. (3) Log pipeline timing per step. (4) Record engine state hash in export audit log.

5. Computation Workflow / 计算 workflow

5.1 Input Data Format / 输入数据格式

Uploaded files are processed by `file_parser.py`. Each file produces a dict: `{text: str, success: bool, format: str, error: str, filename: str}`. The 'text' field is the full plaintext extraction of the file. Required: at least one file with non-empty text content. `papers_df` columns: `filename` (str), `text` (str), `year` (int, extracted via regex from first 2000 chars). `keywords_df` columns after `compute_all`: `keyword`, `tfidf_mean`, `doc_freq`, `first_year`, `centrality`, `pagerank`, `score`, `eco_role`, `niche_level`, `carbon_sink`, `community`.

5.2 Pipeline Steps / 流水线步骤

Step	Name	Function(s)	Details	Complexity
S1	TF-IDF Extraction	<code>extract_keywords_from_texts(raw_texts)</code>	<code>top_n=200</code> , <code>min_df=2</code> , <code>max_df=0.95</code> , <code>stop_words=english</code> , token regex: 2+ letter chars (incl. Chinese). Produces <code>keywords_df</code> .	$O(D *V)$
S2	Co-occurrence Matrix	<code>build_cooccurrence_matrix(raw_texts, kw_list)</code>	For each document, find present keywords, count all pairs. Symmetric integer matrix.	$O(D *K^2)$
S3	Network Graph	<code>build_network(keywords_df, cooccurrence)</code>	Add nodes with <code>tfidf/doc_freq</code> attrs; add edges where <code>cooccur >= min_edge_weight</code> (default 2).	$O(K^2)$
S4	Centrality + PageRank	<code>compute_centrality(G)</code> , <code>compute_pagerank(G)</code>	Betweenness centrality (Brandes), PageRank (power iteration, <code>damping=0.85</code>).	$O(V*E) + O(E*iter)$
S5	Ecology Classification	<code>classify_ecology + compute_niche_levels + compute_carbon_sink</code>	<code>score=tfidf+centrality</code> ; <code>top 1%=top_predator</code> , <code>top 10%=dominant</code> , <code>rest=symbiotic</code> ; 5-level quantile niche; <code>carbon_sink=row_sum(cooccur)</code> .	$O(K)$
S6	Diversity Metrics	<code>compute_shannon_index</code> , <code>compute_monopoly_ratio</code>	Shannon H from <code>tfidf</code> proportions; <code>monopoly = top3_tfidf / total_tfidf</code> .	$O(K)$
S7	Community Detection	<code>detect_louvain_communities(G, resolution)</code>	Louvain (python-louvain) with config <code>resolution</code> (default 1.0). Fallback: <code>greedy_modularity</code> .	$O(E*\log V)$
S8	Evolution Tree	<code>build_evolution_tree(keywords_df, cooccurrence)</code>	BFS from highest-score keyword; max 5 children per node, max depth 5.	$O(K*5^5)$ worst case

5.3 Parameter Reference / 参数参考

Parameter	Default	Range	Description	Impact
<code>top_n</code>	200	1-10000	Max keywords extracted	More = richer analysis, slower
<code>min_df</code>	2	1- D	Min document frequency for TF-IDF	1 = include rare terms
<code>max_df</code>	0.95	0-1	Max document frequency ratio	Lower = exclude very common terms
<code>min_edge_weight</code>	2	1-100	Min co-occurrence for graph edge	Higher = sparser graph

Parameter	Default	Range	Description	Impact
top_predator_percentile	0.01	0-1	Top % for predator role	Smaller = fewer predators
dominant_percentile	0.10	0-1	Top % for dominant role	Larger = more dominants
shannon_warning_threshold	1.5	0-10	Below this triggers warning	Context-dependent
niche_levels	5	2-20	Number of niche tiers	5 is the canonical value
louvain_resolution	1.0	0.1-5	Louvain resolution gamma	Higher = more communities
extinction_decay_threshold	0.80	0-1	Decay ratio for extinction	Not used (dead code)
extinction_years	5	1-20	Lookback window	Not used (dead code)

5.4 Output Products / 输出产物

Per-page exports (via `render_export_panel`): (1) JSON file: serialized engine state or page-specific data. (2) CSV file: `keywords_df` or filtered subset. (3) TXT file: AI analysis result text. (4) ZIP file: all of the above plus `audit_log.txt`. File naming: `aeb_{page_name}_{YYYYMMDD_HHMMSS}.{ext}`. Saved via: `Streamlit st.download_button` (browser download dialog). No server-side persistence; all exports are generated on-demand in memory.

5.5 Performance Bottlenecks / 性能瓶颈

(1) Co-occurrence matrix construction (S2): $O(|D| * K^2)$ -- quadratic in keywords per document. For 200 keywords and 1000 documents, this is ~40M comparisons. Observed: fast for typical academic corpora. (2) Betweenness centrality (S4): $O(V * E)$ -- can be slow for dense graphs. For 200 nodes, typically < 1 second. (3) TF-IDF vectorization (S1): Linear in corpus size; sklearn is C-optimized. (4) Three.js rendering: Client-side; 250-frame force simulation may lag on low-end browsers for >100 nodes. (5) PDF parsing: PyMuPDF is fast; pdfplumber fallback is slower. Large PDFs (>100 pages) may take seconds.

6. FAQ & Troubleshooting / 常见问题与故障排除

FAQ-01

Symptom: Streamlit shows 'ModuleNotFoundError: No module named community'

Cause: python-louvain is not installed. Note: the package name is 'python-louvain' on PyPI, not 'community'.

Fix: pip install python-louvain

Verify: import community; community.best_partition should not raise ImportError.

FAQ-02

Symptom: Uploaded PDF shows 0 keywords extracted

Cause: PyMuPDF and pdfplumber both failed to extract text (scanned/image PDF). file_parser falls back to raw bytes which contain no readable text.

Fix: Use OCR-processed PDFs or convert to text first. Alternatively install pytesseract + poppler for OCR (not included in current requirements).

Verify: Re-upload the OCR'd PDF; keywords_df should be non-empty after loading.

FAQ-03

Symptom: 'No model connected' even after configuring Ollama

Cause: Ollama server is not running or the specified model is not pulled.

Fix: Run 'ollama serve' in a terminal first, then 'ollama pull qwen2.5:7b' (or your model). Check sidebar: models should appear in dropdown.

Verify: The sidebar shows a green checkmark with model name.

FAQ-04

Symptom: GGUF loading fails with CUDA/memory error

Cause: n_gpu_layers=-1 tries to load entire model to GPU VRAM. Insufficient VRAM for the model size.

Fix: Set n_gpu_layers to a smaller number (e.g. 20) in the sidebar, or use n_gpu_layers=0 for CPU-only mode.

Verify: Model loads without error; health check returns OK.

FAQ-05

Symptom: OpenAI API returns '[AI Error: ...]'

Cause: Invalid API key, wrong base_url, model name mismatch, or network issue.

Fix: Verify: (1) API key is correct and has credits. (2) Base URL ends with /v1. (3) Model name matches provider's listing. (4) Network allows outbound HTTPS.

Verify: Health check button returns green with latency.

FAQ-06

Symptom: 3D visualizations show blank/black screen

Cause: Three.js CDN (cdnjs.cloudflare.com) is blocked by corporate firewall or ad blocker.

Fix: Whitelist cdnjs.cloudflare.com in firewall/proxy. Or download three.min.js locally and modify THREEJS_CDN in threejs_components.py.

Verify: 3D scenes render with nodes and interactions.

FAQ-07

Symptom: Chinese characters display as garbled text

Cause: Encoding detection failed for the uploaded file.

Fix: Save your file explicitly as UTF-8 before uploading. Or install chardet (should already be in requirements).

Verify: Re-upload; parsed text shows correct Chinese characters.

FAQ-08

Symptom: Session state lost on page refresh

Cause: By design: Streamlit session_state does not persist across browser refreshes. No server-side storage is implemented.

Fix: This is a known limitation. Export your results (ZIP) before refreshing. For persistence, a database backend would need to be added.

Verify: Not applicable; architectural limitation.

FAQ-09

Symptom: All keywords show first_year=2000

Cause: Code limitation: extract_keywords_from_texts() hardcodes first_year=2000. The per-keyword year extraction from papers is not implemented.

Fix: This is a known TODO in the code (comment: 'default, override with actual data'). To fix: after keyword extraction, iterate papers_df to find earliest year each keyword appears.

Verify: After code fix, keywords_df.first_year shows varying years.

6.2 Minimum Runnable Path / 最小可运行路径

```
# 1. Install (Python 3.9+)
pip install streamlit pandas numpy scikit-learn networkx python-louvain \
  scipy pyyaml plotly PyMuPDF pdfplumber python-docx beautifulsoup4 \
  lxml openpyxl chardet
```

```
# 2. Run
cd aeb-project-v3/
streamlit run app.py
```

```
# 3. Upload any .txt file containing English academic text
# Click 'Start Loading' -> Overview metrics appear
# AI is optional. System works fully without AI model configured.
```

7. Quick Start Guide / 快速入门指南

7.1 Environment Setup / 环境配置

Python 3.9 or later (3.11 recommended). GPU: Optional. Only needed for GGUF model loading with GPU acceleration. CPU works for all features.

```
# Full install (with optional AI backends)
pip install -r requirements.txt
```

```
# Minimal install (core features only, no AI, no PDF/docx parsing)
pip install streamlit pandas numpy scikit-learn networkx python-louvain \
  scipy pyyaml plotly chardet
```

7.2 Launch / 启动

```
# Standard launch
streamlit run app.py
```

```
# Custom port
streamlit run app.py --server.port 7861
```

```
# Windows: double-click run.bat
```

Browser opens automatically at <http://localhost:8501> (or configured port).

7.3 First Run Demo / 首次运行演示

Step 1: In the sidebar, click the file upload area. Upload one or more academic papers (e.g., a .txt file with English research text, or a .pdf paper). Step 2: Click the green 'Start Loading' button. A progress bar shows parsing and computation progress. Step 3: After completion, the main page displays 5 metric cards: Total Papers, Total Keywords, Top Predators, Dominant Species, Extinct Species. Three tabs below show top predators, dominant species, and niche clusters. Step 4: Navigate to any of the 13 pages in the left sidebar. Each page shows its specific visualization immediately (no additional loading needed). Step 5: (Optional) Configure an AI backend in the sidebar bottom panel. Then click 'AI Analyze' on any page for natural language insights.

7.4 Key Page Operations / 关键页面操作

Page	Key Operations
01 Evolution Tree	Drag to rotate, scroll to zoom, hover for tooltip. Adjust 'Tree Depth' slider. Select keyword for detail metrics.
02 Neuron Graph	Set Top N, Min Edge, Color By. Drag/scroll/click on nodes. Click a node to highlight its edges.
03 Fossil Layers	Slide 'Active Layer' to reveal geological strata. Each layer shows keywords from that era.
04 Extinction Sim	Drag year slider to see species status at that point. Donut chart + bar chart update.
05 Fitness Curve	Multi-select keywords. Line chart shows frequency over years with peak annotations.
06 Migration Path	Multi-select keywords. Animated particles travel along co-occurrence paths. Click Pause to freeze.
07 Origin Trace	Select a keyword. System traces back through co-occurring ancestors. Displays ancestry chain + score chart.

Page	Key Operations
08 Carbon Sink	Treemap + horizontal bar show knowledge contribution. Select keyword for neighbor list.
09 Niche Cluster	Scatter plot: TF-IDF vs Centrality, colored by community. Pie chart shows community sizes.
10 Sedimentation	Bar chart: papers per year. Line overlay: rate change. Red bars exceed threshold.
11 Fossil Report	Select keyword for deep-dive: timeline, co-existing species, replacement chain, paper count.
12 Niche Hierarchy	3D layered visualization: L1 (foundation) to L5 (domain). Hover for keyword names.
13 Collection	Create named collections, add keywords, compare via Jaccard overlap, view diversity scores.

7.5 Export / 导出

Every page has an export panel at the bottom with up to 4 buttons: JSON, TXT, CSV, ZIP. Click any to trigger a browser download. ZIP contains all formats plus an audit_log.txt.

7.6 Security Notes / 安全说明

Data stays local: All processing runs in-process within the Streamlit server. No data is sent to external servers UNLESS an OpenAI-compatible API backend is configured (prompts contain keyword statistics, not raw paper text). API keys: Stored in session_state only (memory); not written to disk. Config.yaml may contain API keys if manually entered -- do not commit to version control. Audit log in ZIP exports includes page name and timestamp. No paper content is included in exports unless explicitly in the AI analysis text. Cache: Streamlit's @st.cache_data caches config.yaml parsing; no sensitive data cached.

8. Supplementary Notes / 补充说明

8.1 Dependency Version Pinning

requirements.txt uses `>=` version constraints only. For reproducible deployments, generate a pinned requirements file: `pip freeze > requirements-lock.txt`. Critical: `python-louvain` has had API changes between versions; pin to 0.16 for compatibility with the `import community` pattern.

8.2 Unit Test Coverage (currently zero)

No test files exist in the codebase. Recommended minimum test suite: (1) `test_file_parser.py` -- test each parser with minimal valid input. (2) `test_data_engine.py` -- test TF-IDF extraction, co-occurrence, centrality, classification with a synthetic 3-document corpus. (3) `test_aeb_model.py` -- mock AI backends, test `generate/stream/health_check` error handling. Use `pytest`. Estimated effort: 2-3 hours for basic coverage.

8.3 Dead Code: `detect_extinction`

`data_engine.py` defines `detect_extinction()` and `AEBDataEngine` has `extinct_keywords` and `year_counts` fields, but `compute_all()` never calls `detect_extinction()`. `year_counts` is always `{}`. Page 04 (`extinction_sim.py`) uses a completely different age-based heuristic (`age > 20 = extinct`). Recommendation: Either integrate `detect_extinction()` into `compute_all()` by building `year_counts` from `papers_df` during ingest, or remove the dead code to reduce confusion.

8.4 `first_year` Hardcoding Issue

`extract_keywords_from_texts()` sets `first_year=2000` for all keywords (line: `"first_year": 2000, # default`). The comment says 'override with actual data' but this override never happens. To fix: After keyword extraction in `compute_all()`, iterate `papers_df` to find the earliest year each keyword appears in: for each keyword, scan `papers_df['text']` and record `min(papers_df['year'])`. This would make fossil layers (Page 03) and extinction simulation (Page 04) more meaningful.

8.5 Score Scale Imbalance

The composite score (F-005) adds `tfidf_mean` (typically 0.001-0.05) and `betweenness centrality` (0-1.0 range). In practice, centrality can dominate by 10-100x. Consider normalizing both to [0,1] before addition, or using a weighted combination with configurable `alpha`.

8.6 Extensibility Points

(1) New file format: Add a `_parse_xxx()` function in `file_parser.py` and register the extension in `parse_file()`. (2) New visualization page: Create `pages/NN_name.py` following existing pattern (import engine, check data, render, export). (3) New AI backend: Add a new `Backend` class in `aeb_model.py` with `generate/generate_stream/health_check` methods. (4) New ecological metric: Add computation function in `data_engine.py`, call it in `compute_all()`, add column to `keywords_df`. (5) Custom themes: Modify `chart_theme.py` palettes and `AEB_LAYOUT` dict.

8.7 Performance Monitoring Recommendation

Add timing instrumentation to `compute_all()` steps: import `time`; record elapsed per step in a dict; display in sidebar or export in audit log. This enables users to identify bottlenecks with their specific corpus. Suggested addition in `compute_all()`:

```
import time
timings = {}
for step_name, step_func in steps:
    t0 = time.perf_counter()
    step_func()
    timings[step_name] = round(time.perf_counter() - t0, 3)
self.compute_timings = timings
```

Appendix A: Dependency List / 依赖列表

Package	Version	Category	Purpose
streamlit	>= 1.30.0	Core	Web application framework
pandas	>= 2.0.0	Core	DataFrame operations
numpy	>= 1.24.0	Core	Numerical arrays
scikit-learn	>= 1.3.0	Core	TF-IDF vectorization
networkx	>= 3.1	Core	Graph analysis, centrality, PageRank
python-louvain	>= 0.16	Core	Louvain community detection
scipy	>= 1.11.0	Core	Scientific computing (transitive dep)
PyYAML	>= 6.0	Core	config.yaml parsing
plotly	>= 5.18.0	Core	2D interactive charts
PyMuPDF	>= 1.23.0	Parser	PDF text extraction (primary)
pdfplumber	>= 0.10.0	Parser	PDF text extraction (fallback)
python-docx	>= 1.0.0	Parser	DOCX parsing
beautifulsoup4	>= 4.12.0	Parser	HTML/XML parsing
lxml	>= 4.9.0	Parser	Fast XML parser backend
openpyxl	>= 3.1.0	Parser	Excel file parsing
chardet	>= 5.2.0	Parser	Encoding detection
ollama	>= 0.3.0	AI (opt)	Ollama API client
openai	>= 1.10.0	AI (opt)	OpenAI-compatible API client
llama-cpp-python	>= 0.2.50	AI (opt)	Local GGUF model inference

Appendix B: Configuration Reference (config.yaml) / 配置参考

Key Path	Default	Description
app.title_zh	学术演化生物圈 AEB	Chinese app title
app.title_en	Academic Evolutionary Biosphere	English app title
app.version	2.0.0	App version string
app.default_lang	zh	Default language (zh or en)
app.theme	dark	UI theme (only dark implemented)
model.llama_cpp.enabled	true	Enable GGUF backend
model.llama_cpp.n_ctx	16384	Context window size
model.llama_cpp.n_gpu_layers	-1	GPU layers (-1=all)
model.llama_cpp.default_gguf_path	""	Default GGUF file path
model.ollama.enabled	true	Enable Ollama backend
model.ollama.base_url	http://localhost:11434	Ollama server URL
model.ollama.default_model	qwen2.5:7b	Default Ollama model
model.openai_compatible.enabled	true	Enable OpenAI backend
model.openai_compatible.base_url	https://api.openai.com/v1	API base URL
model.openai_compatible.api_key	""	API key (leave empty for security)
model.openai_compatible.model	gpt-4o-mini	Model name
model.openai_compatible.timeout	120	Request timeout (seconds)
model.openai_compatible.max_retries	3	Auto retry count
model.openai_compatible.stream	true	Enable streaming
ecology.top_predator_percentile	0.01	Top 1% = predator
ecology.dominant_percentile	0.10	Top 10% = dominant
ecology.extinction_decay_threshold	0.80	Decay ratio for extinction (unused)
ecology.extinction_years	5	Lookback window (unused)
ecology.shannon_warning_threshold	1.5	Shannon index warning level
ecology.sedimentation_warning_rate	50	Papers/year warning (Page 10 default)
ecology.niche_levels	5	Number of niche tiers
ecology.louvain_resolution	1.0	Louvain resolution parameter
export.png_scale	2	PNG export scale factor (unused in code)
export.default_format	json	Default export format (unused in code)

Appendix C: i18n Key Reference (partial) / 国际化键名参考

The i18n.py module contains 60+ bilingual key-value pairs in the TEXTS dictionary. Each key maps to {zh: '...', en: '...'}. Access via t('key_name', param1=value1). Language is toggled at runtime via toggle_lang() in sidebar. Below are the most frequently used keys:

Key	Chinese (zh)	English (en)
app_title	学术演化生物圈 AEB	Academic Evolutionary Biosphere
sidebar_title	AEB 控制面板	AEB Control Panel
upload_title	上传学术论文	Upload Academic Papers
start_loading	开始加载	Start Loading
no_data	暂无数据，请先上传论文并开始加载。	No data yet. Please upload papers and start loading.
total_papers	论文总数	Total Papers
total_keywords	关键词总数	Total Keywords
top_predators	顶级掠食者	Top Predators
shannon_index	Shannon多样性指数	Shannon Diversity Index
monopoly_ratio	垄断比率	Monopoly Ratio
run_ai	AI 分析	AI Analyze
eco_collapse_warning	生态系统崩溃风险：{kw}关键词过度垄断	Ecosystem Collapse Risk: {kw} keyword over-monopoly